COMP 3522

OBJECT ORIENTED PROGRAMMING 2

Midterm Exam

February 2024

Full Name	
Signature	
Student ID	
Set	

Instructions:

1. No phones. Turn off your phone and put it away. If your phone rings you will earn zero on this exam.
2. This is a closed-book exam. One cheat sheet allowed. You may not use any notes, texts, manuals, etc., during the exam
3. No electronic devices. No calculators. No computers. No pocket organizers or translators. No watches. No music players. No headphones.
4. No talking. Speaking during the exam is forbidden. If you speak, you will earn zero.
5. Keep your eyes on your own paper. Looking at another exam, or purposely exposing your exam to the view of other candidates will be considered cheating and you will earn zero.
6. This midterm is 90 minutes long.
7. You must write your answers on this exam. You may write on the back.
8. No candidate will be admitted after 30 minutes.
9. No candidate will be permitted to leave during the first 30 minutes.
10. NO QUESTIONS. Candidates are NOT permitted to ask questions during the exam except in the case of supposed typos.
11. Add The answers for T/F and MCQ questions to the Answer sheet in Page 2.
12. Good luck. You're awesome, you've got this.

Final grade:

Name		Date	
Class		Quiz	

Student ZipGrade ID

Key Version
A
B
C
D
E

						A	B	C	D	E		A	B	C	D	E		A	B	C	D	E
1						11						31										
2						12						32										
3						13						33										
4						14						34										
5						15						35										
6						16						36										
7						17						37										
8						18						38										
9						19						39										
0						20						40										

	A	B	C	D	E		A	B	C	D	E		A	B	C	D	E
1						21						41					
2						22						42					
3						23						43					
4						24						44					
5						25						45					
6						26						46					
7						27						47					
8						28						48					
9						29						49					
10						30						50					

Students: Fill circles completely with black ink or pencil.
Erase all stray marks completely.

Teachers: Hold paper on flat surface when grading.
Be aware of bright lights and shadows.

ZIPGRADE.COM

True/False Questions

[.5 Mark for each] [Total of 4 Marks]

Mark the following statements as True[T] or False[F].

1. [ ] Dictionaries in Python can have immutable objects as keys, such as tuples.
2. [ ] The timedelta class in the datetime module is used to perform arithmetic operations on datetime objects.
3. [ ] repr Function is meant for display purposes and should be used when you want a string that is informative and readable to humans whereas str function is meant for debugging and development.
4. [ ] In Python, the interpreter ensures that operations are performed on variables of compatible types because Python is a statically typed language.
5. [ ] For the following code, both Duck and RobotDuck are nominally the same type.

```
class Duck:
    def quack(self):
        print("Quack!")

class RobotDuck:
    def quack(self):
        print("Robo-quack!")
```

```
def make_sound(entity):
    entity.quack()

duck_instance = Duck()
robot_duck_instance = RobotDuck()

make_sound(duck_instance)
make_sound(robot_duck_instance)
```

6. [ ] The following code compliant with the Open Closed principle

```
class DocumentProcessor:
    def process(self, document):
        if document.type == "XML":
            # Process XML document
            pass
        elif document.type == "json":
            # Process JSON document
            pass
        elif document.type == "html":
            # Process HTML document
            pass
```

```
class XML(DocumentProcessor):
    def __init__(self, document):
        self.type = "XML"

class JSON(DocumentProcessor):
    def __init__(self, document):
        self.type = "json"

class HTML(DocumentProcessor):
    def __init__(self, document):
        self.type = "html"
```

7. [ ] Unpacking characters using the (*) operator is a convenient way to pass each character as a separate argument to a function.
8. [ ] A realization relationship in UML is used to represent the implementation of an interface by a class, indicating that the class provides the behavior specified by the interface.

MCQ Questions

[10 Questions. 1 Mark for each. 10 Marks in Total]

1. What is the output of the following code?

```
def format_username(name):
    cleaned_name = name.strip()

    lowercase_name = cleaned_name.lower()

    formatted_name = lowercase_name.replace(" ", "_")

    return formatted_name

user_input = input("Enter your full name: ")

username = format_username(user_input)

print("Formatted username:", username)
```

A.

```
Enter your full name:   John Doe
Formatted username: john doe
```

B.

```
Enter your full name:   John Doe
Formatted username:   john Doe
```

C.

```
Enter your full name:   John Doe
Formatted username:   john_doe
```

D.

```
Enter your full name:   John Doe
Formatted username: john_doe
```

2. What is the output of the following code?

```
class MutableObject:
    def __init__(self, value):
        self.value = value


def rebind_object(obj):
    obj = MutableObject(200)


my_object = MutableObject(100)

print("Original Value:", my_object.value)

rebind_object(my_object)

print("After Rebinding, Value:", my_object.value)
```

A.

```
Original Value: 100
After Rebinding, Value: 100
```

B.

```
Original Value: 100
After Rebinding, Value: 200
```

3. Which of the following best describes the relationship between the Person and Address classes in the code below?

```
class Address:
    def __init__(self, street, city, state, zip_code):
        self.street = street
        self.city = city
        self.state = state
        self.zip_code = zip_code

    def display_address(self):
        return f"{self.street}, {self.city}, {self.state} {self.zip_code}"


class Person:
    def __init__(self, name, age, address):
        self.name = name
        self.age = age
        self.address = address

    def display_person_info(self):
        return f"Name: {self.name}, Age: {self.age}, Address: {self.address.display_address()}"


home_address = Address("123 Main St", "Anytown", "CA", "12345")

person1 = Person("John Doe", 25, home_address)

print(person1.display_person_info())
```

A. Composition

B. Association

C. Aggregation

D. Dependency

4. Which of the following SOLID principles best describes the transformation from the non-compliant code to the compliant code in the snippet below?

```python
# non-compliant version
class PaymentProcessor:
    def process_payment(self, amount):
        # Logic to process payment
        pass

class OrderService:
    def __init__(self):
        self.payment_processor = PaymentProcessor()

    def checkout(self, order_total):
        self.payment_processor.process_payment(
order_total)

# Usage
order_service = OrderService()
order_service.checkout(100)
```

```python
# Compliant version
class PaymentProcessor:
    def process_payment(self, amount):
        pass

class PayPalProcessor(PaymentProcessor):
    def process_payment(self, amount):
        # Logic to process payment via PayPal
        pass

class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount):
        # Logic to process payment via Stripe
        pass

class OrderService:
    def __init__(self, payment_processor):
        self.payment_processor = payment_processor

    def checkout(self, order_total):

self.payment_processor.process_payment(order_tot
al)

# Usage
paypal_processor = PayPalProcessor()
order_service_paypal =
OrderService(paypal_processor)
order_service_paypal.checkout(100)

stripe_processor = StripeProcessor()
order_service_stripe =
OrderService(stripe_processor)
order_service_stripe.checkout(200)
```

A. Single Responsibility Principle

B. Dependency Inversion Principle

C. Open/Closed Principle

D. Demeter Law

5. Which of the following code snippets is compliant with the Law of Demeter (LoD)?

A.

```
class Owner:
   def __init__(self):
      self.pet = Dog()

   def askPetToFetch(self, ball):
      self.pet.fetch(ball)

class Dog:
   def fetch(self, ball):
      print("Fetching the ball")

class Ball:
   pass

owner = Owner()
owner.askPetToFetch(Ball())
```

B.

```
class Owner:
   def __init__(self):
      self.pet = Dog()

class Dog:
   def fetch(self, ball):
      print("Fetching the ball")

class Ball:
   pass

owner = Owner()
owner.pet.fetch(Ball())
```

6. What is the range of afternoon and evening hours in the following code?

```
# Sample dataset: Temperature readings for each hour of a 24-hour day
temperature_data = [12, 14, 14, 14, 16, 16, 20, 22, 24, 24, 26, 26,
                    28, 26, 26, 24, 20, 18, 16, 16, 14, 12, 12, 12]

morning_temperatures = temperature_data[:6]
afternoon_temperatures = temperature_data[6:12]
evening_temperatures = temperature_data[17:21]
```

A. Afternoon: 20 to 26, Evening: 18 to 12

B. Afternoon: 20 to 28, Evening: 18 to 14

C. Afternoon: 20 to 26, Evening: 18 to 14

D. Afternoon: 20 to 28, Evening: 18 to 12

7. What is the output of the following code?

```
students = [
   {'name': 'Alice', 'age': 22, 'major': 'Computer Science', 'gpa': 3.8},
   {'name': 'Bob', 'age': 20, 'major': 'Mathematics', 'gpa': 3.5},
   {'name': 'Charlie', 'age': 21, 'major': 'Physics', 'gpa': 3.9},
   {'name': 'David', 'age': 22, 'major': 'Computer Science', 'gpa': 3.7},
   {'name': 'Eve', 'age': 23, 'major': 'Biology', 'gpa': 3.6},
]

cs_major_high_gpa_names = [student['name'] for student in students if student['major'] == 'Computer Science' and student['gpa'] > 3.7]
```

A.

```
['Alice', 'David']
```

B.

```
['Alice']
```

C.

```
[{'name': 'Alice', 'age': 22, 'major': 'Computer Science', 'gpa': 3.8}]
```

D.

```
[{'name': 'Alice', 'age': 22, 'major': 'Computer Science', 'gpa': 3.8},
 {'name': 'David', 'age': 22, 'major': 'Computer Science', 'gpa': 3.7}]
```

8. What is the output of the following code?

```
class InvalidInputError(Exception):
    def __init__(self, value, message="Invalid input"):
        self.value = value
        self.message = message
        super().__init__(self.message)

def calculate_square_root(number):
    try:
        if number < 0:
            raise InvalidInputError(
                number, "Cannot calculate square root of a negative number.")
        result = number ** 0.5
    except InvalidInputError as e:
        print(f"{e.message}: {e.value}")
    else:
        print(f"The square root of {number} is: {result}")
    finally:
        print("Square root calculation process completed")

# Example 1: Calculating square root of a negative number
calculate_square_root(-25)

# Example 2: Calculating square root of a positive number
calculate_square_root(64)
```

A.

```
Cannot calculate square root of a negative number.: -25
Square root calculation process completed
The square root of 64 is: 8.0
Square root calculation process completed
```

B.

```
Cannot calculate square root of a negative number.: -25
Square root calculation process completed
```

C.

```
Cannot calculate square root of a negative number.: -25
The square root of 64 is: 8.0
Square root calculation process completed
```

D.

```
Cannot calculate square root of a negative number.: -25
Square root calculation process completed
NameError: name 'result' is not defined
Square root calculation process completed
```

9. Which of the following generator expressions is equivalent to the following generator function?

```
def power_generator(base, exponent_limit):
    exponent = 0
    while exponent <= exponent_limit:
        yield base ** exponent
        exponent += 1


# Using the generator to print powers of 3 up to exponent 4
base_value = 3
exponent_limit = 4
power_gen = power_generator(base_value, exponent_limit)

for result in power_gen:
    print(result)
```

A.

```
power_gen = {base_value ** exponent for exponent in range(exponent_limit + 1)}
```

B.

```
power_gen = (base_value ** exponent for exponent in range(exponent_limit))
```

C.

```
power_gen = {base_value ** exponent for exponent in range(exponent_limit + 1)}
```

D.

```
power_gen = (base_value ** exponent for exponent in range(exponent_limit + 1))
```

10. With the provided code and profiling output, what is the execution time for each call to the ackermann(3, 6) function?

```python
import sys

def ackermann(m, n):
    if m == 0:
        return n + 1
    elif m > 0 and n == 0:
        return ackermann(m - 1, 1)
    elif m > 0 and n > 0:
        return ackermann(m - 1, ackermann(m, n - 1))
```

```python
def main():
    result = ackermann(3, 6)
    result2 = ackermann(3, 6)
    print(result)
    print(result2)


if __name__ == "__main__":
    current = sys.getrecursionlimit()
    print(f"recursion limit {current}")
    sys.setrecursionlimit(1000)

    main()
```

```
python -m cProfile -s name profile_ackermann.py
```

Output:

```
recursion limit 1000
509
509
         344475 function calls (11 primitive calls) in 0.116 seconds

   Ordered by: function name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.116    0.116 {built-in method builtins.exec}
        3    0.001    0.000    0.001    0.000 {built-in method builtins.print}
        1    0.000    0.000    0.000    0.000 {built-in method sys.getrecursionlimit}
        1    0.000    0.000    0.000    0.000 {built-in method sys.setrecursionlimit}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        1    0.000    0.000    0.116    0.116 profile_ackermann.py:1(<module>)
344466/2    0.115    0.000    0.115    0.058 profile_ackermann.py:4(ackermann)
        1    0.000    0.000    0.115    0.115 profile_ackermann.py:13(main)
```

A. 0.116 seconds

B. 0.115 seconds

C. 0.058 seconds

D. 0.029 seconds

What is the output?

[1 Marks for each part] [Total of 2 Marks]

A) Complete the following computations for the linearization of the following class hierarchy?

```
class X:pass
class Y:pass
class Z:pass
class A(X,Y):pass
class B(Y,Z):pass
class M(B,A,Z):pass
```

L(X) = [X]
L(Y) = [Y]
L(Z) = [Z]
L(A) = [A, X, Y]
L(B) = [B, Y, Z]
L(M) = [M, B, A, Z]
L(M) = [M] + merge(L(B), L(A), L(Z), [B, A, Z])

B) What is the output of the following code?

```
def generate_department_report(department_name, employee_count):
    header = f"=== {department_name[:7]} Report ===\n"

    summary = f"Employees: {employee_count} \n"

    separator_line = "=" * 14 + "\n"

    full_report = header + separator_line + summary + separator_line

    return full_report


department1 = ("Sales", 50)
department2 = ("Engineering", 75)

report1 = generate_department_report(*department1)
report2 = generate_department_report(*department2)

print(report1)
print(report2)
```

In one sentence, write the definition for the following terms

[.5 Mark for each Part] [Total of 1 Mark]

1. What is an enum?

2. What is a generator?

Writing Code

[8 Marks]

The following code defines a simple clock system in Python. It consists of three classes: ClockTimer, DigitalClock, and AnalogClock. The ClockTimer class serves as the central clock controller, managing time in seconds and notifying attached clocks of time changes. DigitalClock and AnalogClock are classes representing different types of clocks.

The ClockTimer class initializes with a specific time in the format “HH:MM:SS” and allows for the increment of time through the tick method. It maintains a list of attached clocks and notifies them to update their displays when the time changes. The DigitalClock class displays time in the format “HH:MM:SS”, while the AnalogClock class represents time with two hands, a long hand for hours and a short hand for minutes.

The draw methods in both DigitalClock and AnalogClock are responsible for visualizing the current time. The example usage at the end creates an instance of ClockTimer with an initial time of “12:00:00”, attaches a DigitalClock and an AnalogClock to it, and then simulates the passage of time with three consecutive calls to the tick method, updating the displays of both clocks accordingly.

```
import abc


class ClockTimer:
    """
    A ClockTimer class that keeps track of the time and notifies all the clocks when the time changes.
    """

    def __init__(self, time):
        """
        Initialize the clock with the current time.
        :param time: a string in the format "HH:MM:SS"
        This time is in military time format. It starts at 00:00:00. and goes to 23:59:59.
        12:00:00 is noon and 00:00:00 is midnight.
        """
        self._time_in_seconds = (
            int(time.split(":")[0]) * 3600
            + int(time.split(":")[1]) * 60
            + int(time.split(":")[2])
        )
        self.clocks = []

    def tick(self):
        """
        Increment the time by one second and notify all the clocks.
        """
        self._time_in_seconds += 1
        self.draw()
```

```
    def draw(self):
        """
        Draw the clock. The clock can be a digital clock or an analog clock. The clock should be drawn with the current
time.
        """
        for clock in self.clocks:
            if clock.__class__.__name__ == "DigitalClock":
                clock.draw(self.get_hour(), self.get_minute(), self.get_second())
            elif clock.__class__.__name__ == "AnalogClock":
                clock.draw(self.get_hour(), self.get_minute())
            else:
                raise Exception("Unknown clock type")

    def get_hour(self):
        # logic to get the hours from self._time_in_seconds

    def get_minute(self):
        # logic to get the minutes from self._time_in_seconds

    def get_second(self):
        # logic to get the seconds from self._time_in_seconds

    def add_clock(self, clock):
        self.clocks.append(clock)


class DigitalClock:
    """
    A digital clock that displays the time in the format HH:MM:SS
    """

    def __init__(self):
        pass

    def draw(self, hours, minutes, seconds):
        """
        Draw the digital clock. Digital clocks are represented in the format HH:MM:SS
        """
        print(f"DigitalClock: {hours}:{minutes}:{seconds}")


class AnalogClock:
    """
    An analog clock that displays the time with two hands, a long hand and a short hand.
    """

    def __init__(self):
```

```
        pass

    def draw(self, hours, minutes):
        """
        Draw the analog clock. Analog clocks are represented with two hands, a long hand and a short hand.
        """
        print(f"AnalogClock: Long Hand->{hours}, Short Hand->{minutes}")
```

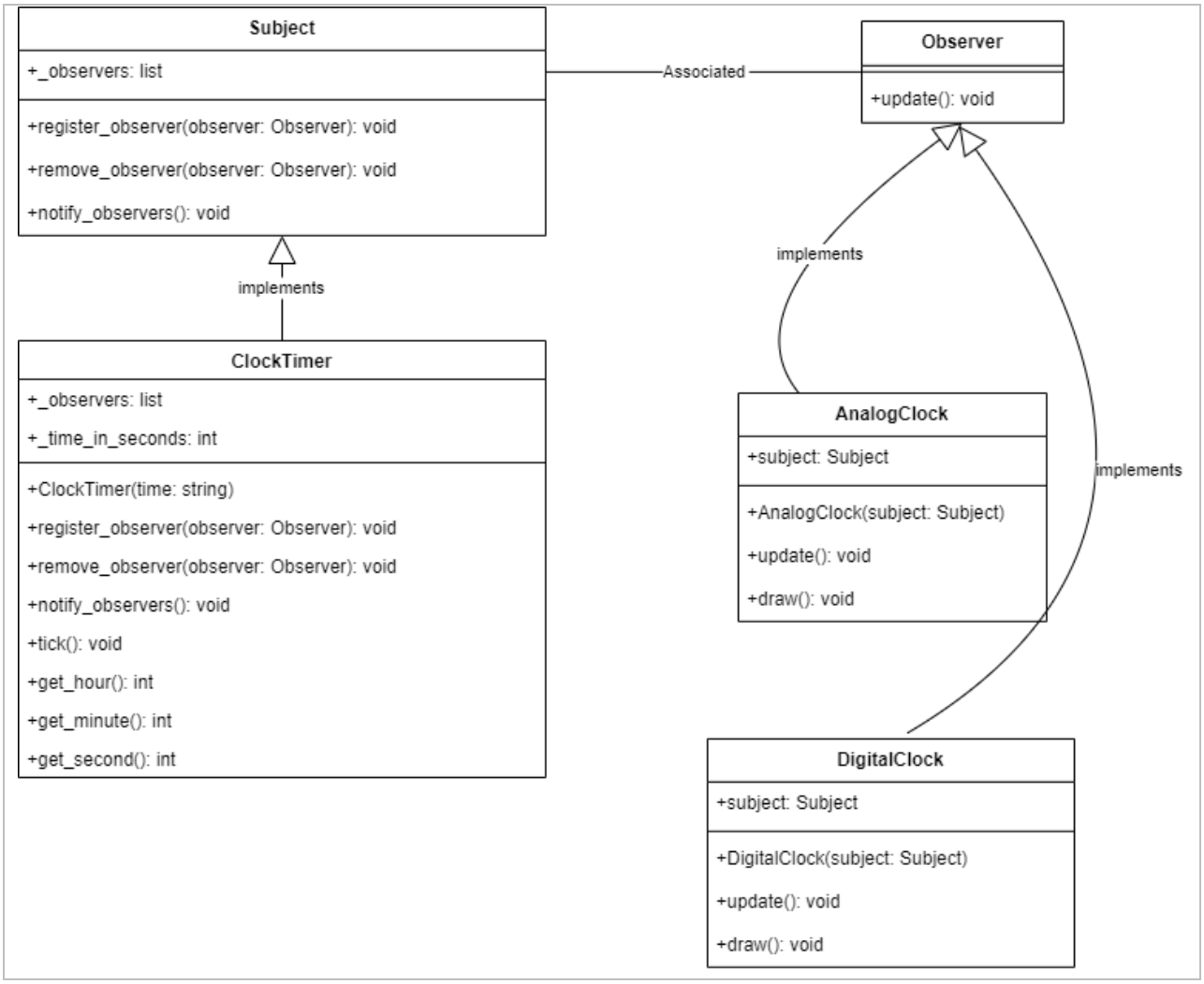
```
# Usage
timer = ClockTimer("12:00:00")
digital_clock = DigitalClock()
analog_clock = AnalogClock()
timer.add_clock(digital_clock)
timer.add_clock(analog_clock)

timer.tick()
timer.tick()
timer.tick()
```

Output

```
DigitalClock: 12:0:1
AnalogClock: Long Hand->12, Short Hand->0
DigitalClock: 12:0:2
AnalogClock: Long Hand->12, Short Hand->0
DigitalClock: 12:0:3
AnalogClock: Long Hand->12, Short Hand->0
```

Rewrite the previous code using the Observer design pattern. The ClockTimer class should now have a list of observers that it notifies when the time changes. The DigitalClock and AnalogClock classes should be observers that update their displays when notified by the ClockTimer. The example output should remain the same.

Before you start, take a look at the UML diagram below to understand the relationships between the classes.



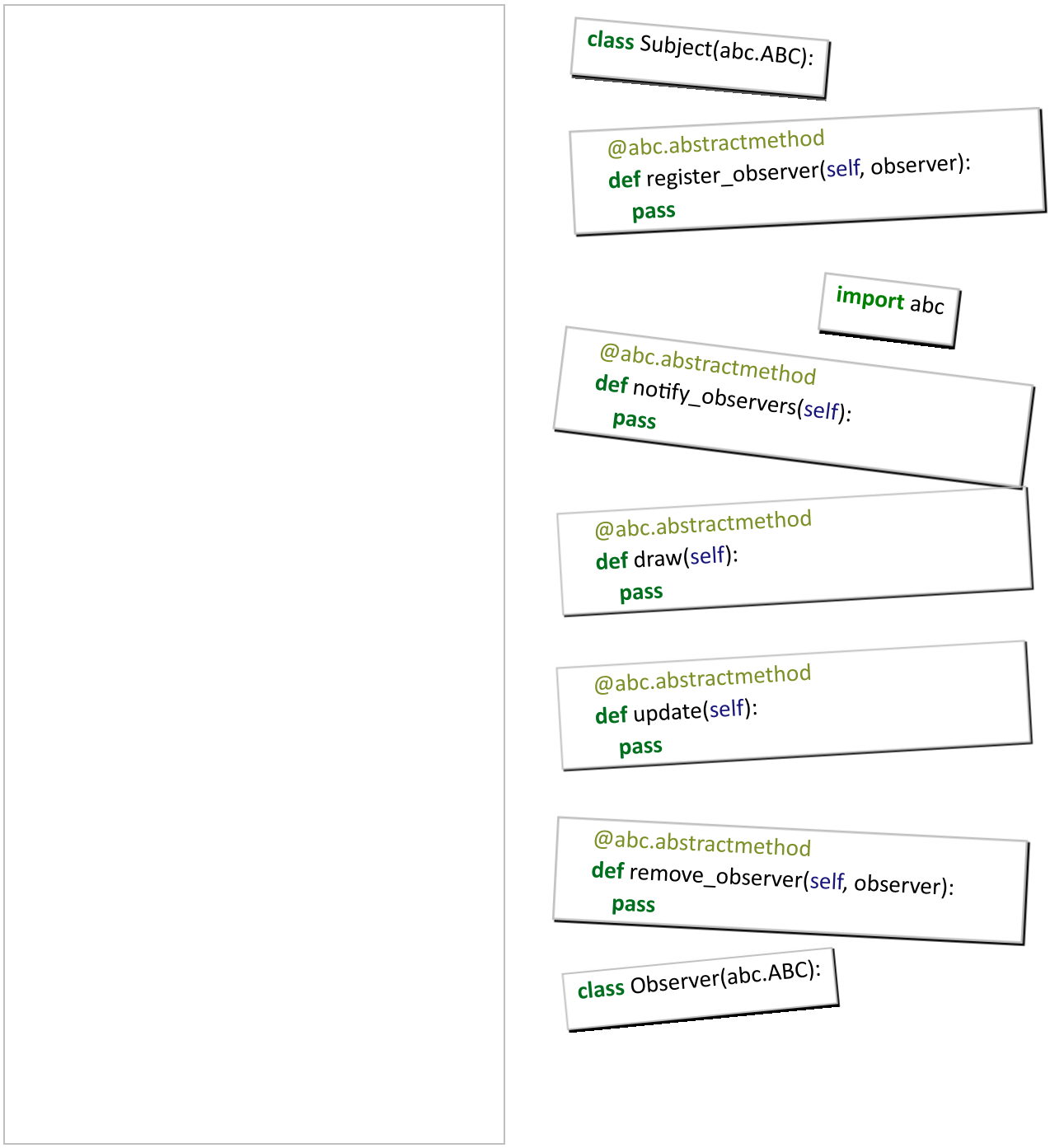
Also, here is the usage of the classes. It is a **moreless** the same as before.

```
# Usage
timer = ClockTimer("12:00:00")
digital_clock = DigitalClock(timer)
analog_clock = AnalogClock(timer)

timer.tick()
timer.tick()
timer.tick()
```

Step 1- Define the Subject and Observer Interfaces [2 Marks]

Complete the following code to define the Subject and Observer interfaces. The Subject interface should have methods to register, remove, and notify observers. The Observer interface should have a method to update the observer when notified by the subject. **The update method should take no arguments.**

Reconstruct the code snippets to make it work. Write the code in the box on the left. Alternatively, you may use arrows to reorganize the snippets.



Step 2- Modify the ClockTimer Class [3 Marks]

Modify the ClockTimer class to implement the Subject interface. The ClockTimer class should maintain a list of observers and provide methods to register, remove, and notify observers. The tick method should notify all the observers when the time changes.

Use the code snippets to make it work. Write the code in the boxes on left. You may use arrows to connect the snippets to the boxes.

```
class ClockTimer(Subject):
    def __init__(self, time):
        [          ]
        # time is a string in the format "HH:MM:SS" converted to seconds
        self._time_in_seconds = (
            int(time.split(":")[0]) * 3600
            + int(time.split(":")[1]) * 60
            + int(time.split(":")[2])
        )

    def register_observer(self, observer):
        [          ]

    def remove_observer(self, observer):
        [          ]

    def notify_observers(self):
        [          ]

    def tick(self):
        [          ]

    def get_hour(self):
        return [          ]

    def get_minute(self):
        return [          ]

    def get_second(self):
        return [          ]
```

self.notify_observers()

self._observers = []

(self._time_in_seconds % 3600) // 60

self._observers.append(observer)

for observer in self._observers:
    observer.update()

self._observers.remove(observer)

(self._time_in_seconds // 3600)

self._time_in_seconds += 1

self._time_in_seconds % 60

Step 3- Modify the DigitalClock and AnalogClock Classes [3 Marks]

Modify the DigitalClock and AnalogClock classes to implement the Observer interface. In the Initialize method, the DigitalClock and AnalogClock classes should register themselves as observers with the ClockTimer class. The update method should update the display of the clock. The update mehtod takes no arguments but make the call to get_hour, get_minute, and get_second methods of the ClockTimer class.

Use the code snippets to make it work. You may use the same snippet more than once. Write the code in the boxes on the left. You may use arrows to connect the snippets to the boxes.

```
class DigitalClock(Observer):
    def __init__(self, subject):
        [          ]


    def update(self):
        [          ]


    def draw(self):
        # padd the minue and hours with 0 if they are
less than 10
        hours = str(self.hours).zfill(2)
        minutes = str(self.minutes).zfill(2)
        seconds = str(self.seconds).zfill(2)
        print(f"DigitalClock:
{hours}:{minutes}:{seconds}")

class AnalogClock(Observer):
    def __init__(self, subject):
        [          ]


    def update(self):
        [          ]


    def draw(self):
        print(
            f"AnalogClock: Long Hand->{self.hours}, Short
Hand->{self.minutes}"
        )
```

self.minutes = str(self.subject.get_minute())

self.subject.register_observer(self)

self.draw()

self.hours = str(self.subject.get_hour())

self.subject = subject

self.seconds = str(self.subject.get_second())

END OF EXAM